CHAPTER 5: SYSTEM IMPLEMENTATION, TESTING AND EVALUATION

> **Status of this file.** Report-ready content for Chapter 5, produced from a read-only investigation of the `student-beneficiary-system` repository (branch `fix/sbms-completion-work`, commit `d6b5032`), its test suites, its execution records and its deployment log.
>
> **Evidence provenance is stated for every result, and three kinds are never merged:**
>
> 1. **Implementation evidence** — read from source, configuration, migrations or the running artefact.
> 2. **Recorded execution evidence** — a test run written into project records by a previous session, with date, commit and command recorded there. Cited, not re-run in this documentation pass.
> 3. **Results not obtained** — marked **NOT RUN**, listed in §5.4.12, and never described as a pass.
>
> No test result, metric, screenshot or participant was invented. Screenshot placeholders name what the image must show; they do not claim an image exists.

5.1 Introduction

This chapter reports how the design in Chapter Four was implemented, how it was tested, and what the evidence permits the project to claim. It has three purposes.

First, it documents the implementation: the environment in which the system was built, the module-by- module realisation of each requirement, and the specific decisions that carry the system's behaviour — password hashing, token handling, batching of model calls, aggregation of per-pair scores into review cases, the two payment gates, the certificate snapshot, and the guarded start-up sequence.

Second, it documents the testing actually performed, at the level of individual test cases where the suites are available, and separates four kinds of confidence: automated unit and integration tests against a real PostgreSQL database, service-level tests of the Python predictor, browser-driven checks in headless Chrome, and checks performed against the live Azure deployment. A feature that exists but has no executed test is stated as such.

Third, it evaluates the system against the objectives in Chapter One and the requirements in Chapter Four, including a full treatment of the machine-learning component: training data, classification performance on a synthetic holdout, the consequence of the operating point for queue volume, and the reasons why none of those figures is an accuracy claim about Kenyan beneficiary records.

The chapter does not argue that the system is complete or production-ready. It reports what works, what was measured, and what remains unverified, so that Chapter Six can draw conclusions that follow from evidence rather than from intent.

5.2 Implementation environment

5.2.1 Hardware

The hardware on which the project was developed and tested **was not recorded** in the repository, and the workstation specification must be supplied by the author before submission. What the environment did record is material to interpreting results: the development host had approximately **7.4 GB of total memory with about 0.5 GB free** during long test sweeps, which is the documented cause of the intermittent failures described in §5.4.3. Claiming a processor model, memory size or storage type would be invention. **REQUIRES INPUT** (gap register item G-01).

5.2.2 Software, frameworks and versions

Table 5.1: Implementation environment and version evidence

Category	Component	Version	Evidence
Language (frontend)	JavaScript (ES modules), JSX	—	`frontend/src/**`
Language (backend)	JavaScript (CommonJS) on Node.js	Node 22 in the image (`node:22-bookworm-slim`); Node 22.11.0 on the development host	`Dockerfile:1`; `docs/FINAL_TEST_REPORT.md` header
Language (ML)	Python	3.11.9 on the development host	`docs/FINAL_TEST_REPORT.md` header
Frontend framework	React, React DOM	18.3.1	`frontend/package.json`
Routing	react-router-dom	7.18.3	`frontend/package.json`
Styling	Tailwind CSS (`@tailwindcss/vite`)	4.3.3	`frontend/package.json`
Charts	chart.js / react-chartjs-2	4.5.1 / 5.3.1	`frontend/package.json`
Icons	lucide-react	1.55.0	`frontend/package.json`
HTTP client	axios	1.20.0	`frontend/package.json`
Bundler	Vite (`@vitejs/plugin-react`)	5.4.1	`frontend/package.json`
Backend framework	Express	4.19.2	`backend/package.json`
Validation	zod	4.5.4	`backend/package.json`
Authentication	jsonwebtoken / bcrypt	9.0.3 / 6.0.0	`backend/package.json`
Database driver	pg	8.12.0	`backend/package.json`
QR generation	qrcode	1.5.4	`backend/package.json`
PDF generation	pdfkit	0.20.2	`backend/package.json`
Test runner (backend)	Jest / Supertest	30.5.1 / 7.2.2	`backend/package.json`
Database	PostgreSQL	17.11 locally; PostgreSQL 16 on the managed Azure server	`docs/FINAL_TEST_REPORT.md` §5; `docs/azure-deployment-log-2026-10-31.md` D4
ML libraries	pandas / scikit-learn / xgboost / jellyfish	3.0.5 / 1.9.0 / 3.2.0 / 1.2.1	`ml-service/requirements.txt`
Container base	Debian Bookworm slim	—	`Dockerfile`
Container tooling	Docker Desktop / CLI	Desktop 4.97.0, CLI 29.9.2 (engine availability varied)	`docs/azure-deployment-log-2026-10-31.md` §2; `docs/FINAL_TEST_REPORT.md` §1
Version control and CI	Git, GitHub Actions (`build-push-action` v6, `login-action` v3, `setup-buildx-action` v3)	—	`.github/workflows/publish-api-image.yml`
Cloud platform	Azure Container Apps, App Service (F1), PostgreSQL Flexible Server, region UAE North	—	`docs/azure-deployment-log-2026-10-31.md` §1

Table 5.2: Why each principal technology was used in this project

Technology	Project-specific reason
React single-page application	The proposal required a browser-accessible system for officers on varied devices with no local installation (proposal §1.4). The component model maps one-to-one onto the distinct operational screens, and the two public verification pages had to be routes on the same build
Express REST API	The API serves two very different clients — an authenticated role-gated staff application and two unauthenticated single-purpose endpoints — so explicit routing with per-route middleware is the natural structure
PostgreSQL	The integrity rules are financial: awards, payments, certificates and review events need foreign keys and `CHECK` constraints that the database enforces even if a controller is wrong

Technology	Project-specific reason
Python with scikit-learn and XGBoost	The scorer is a trained classifier with a persisted artefact; Python is where feature engineering, training and evaluation tooling exists, and the same feature function is reused for serving
Jaro–Winkler via `jellyfish`	The problem is near-duplicate names created by spelling variation and reordering; Jaro–Winkler rewards shared prefixes and tolerates transposition, and its normalised score is directly usable as a feature
`qrcode` and `pdfkit`	QR images and printable documents must be produced server-side so that the token and the document content are authoritative rather than client-generated
Azure (Container Apps, App Service, PostgreSQL Flexible Server)	The deployment had to run inside an Azure for Students credit with a spending limit, in the only regions the subscription policy permitted; the chosen topology is the cheapest combination that still provides HTTPS origins for QR scanning

5.2.3 Development and operations tooling

Tool	Use in this project
`nodemon` + `concurrently`	Runs the API and Vite dev server together from the repository root (`npm start`)
Vite dev server	Serves the frontend on port 4174 and proxies `/api` to port 5000, which is why relative API calls work in development
Jest with Supertest	Drives the real Express app over real HTTP against a real PostgreSQL database
`node --test`	Runs seven self-contained native test files (readiness, start-up, bootstrap, error handling, verification, migration and secret validation) in-process
Python `unittest`	Runs the ML service suite and the research-helper suite; `pytest` is not installed in the virtual environment
`scripts/browser-smoke.mjs`, `scripts/qr-smoke.mjs`, `scripts/admin-reveal-smoke.mjs`, `scripts/registration-fields-smoke.mjs`	Drive headless Chrome over the DevTools Protocol using only Node 22's built-in `WebSocket`, with no additional dependency
`backend/scripts/migrate.js`	Applies migrations in filename order and records them in `sbms_schema_migrations`
`backend/scripts/provision-test-database.js`, `seed-verification-accounts.js`, `seed-kenyan-demo-data.js`, `reset-kenyan-demo-data.js`	Create a disposable test database and synthetic demonstration accounts and records; the verification seeding refuses any non-loopback host
Docker	Builds the single API+predictor image; the build itself runs an inference gate so an image with a broken artefact cannot be published
GitHub Actions (`workflow_dispatch`)	Builds and pushes `ghcr.io/eggxellent/sbms-api` on demand and re-runs the model, feature-order and secret gates against the pushed image
Azure CLI	Provisions the resource group, Container Apps environment, container app, App Service plan and site, and PostgreSQL server

5.2.4 Deployment environment

Two configurations were used and must not be conflated.

Development configuration. API on `http://localhost:5000`, frontend on `http://localhost:4174`, a local PostgreSQL instance, and the predictor from `ml-service/.venv`. Because `PUBLIC_APP_URL` validation deliberately refuses non-loopback local addresses and any plain-HTTP origin outside an explicit `NODE_ENV=development` loopback (Chapter Four §4.2.5), the development configuration can produce QR images but cannot support scanning them with a phone.

Deployed configuration. Verified live on 31 October 2026 and brought back up on 1 November 2026 (`docs/azure-deployment-log-2026-10-31.md` §1, §7): API container app `ca-sbms-api.ambitiousstone-f2de8f67.uaenorth.azurecontainerapps.io`, frontend `https://sbms-web.azurewebsites.net`, PostgreSQL Flexible Server

`pg-sbms` holding database `sbms`, with `PUBLIC_APP_URL` and `CORS_ORIGINS` both set to the frontend origin so that generated QR codes point at a reachable public page.

5.3 System implementation

5.3.1 Code organisation

The repository separates the three tiers into `frontend/`, `backend/` and `ml-service/`, with a root `package.json` that starts the first two together. Two directories carry the project's evidence: `backend/tests/` holds twenty-four Jest suites and `backend/test/` holds the seven native test files plus the out-of-process API smoke test, while `ml-service/tests/`, `ml-service/evaluation/` and `ml-service/experiments/` hold the Python tests and the artefacts of the model work. `docs/` holds the project's own records, and `deploy/` holds the (git-ignored) Azure secrets and operator scripts.

Table 5.3: Implemented modules and their realisation

Module	Requirement(s)	Backend realisation	Frontend realisation	Persistence
Authentication	FR-01 to FR-04	`auth.controller.js`, `auth.model.js`, `middleware/auth.js`, `requireRole.js`, `loginRateLimit.js`	`pages/Login.jsx`, `context/AuthContext.jsx`, `services/api.js`	`users`
Beneficiary registration	FR-05 to FR-11	`students.controller.js`, `student.model.js`, `studentRecordFormat.js`	`pages/StudentRegistration.jsx`, `pages/Students.jsx`	`students`, `duplicate_flags`, `audit_logs`
Duplicate screening	FR-07 to FR-11	`duplicateCheck.js`, `mlDuplicateReview.js`, `duplicateAggregation.js`, `duplicateFlags.controller.js`	`pages/DuplicateReview.jsx`	`duplicate_flags`, `duplicate_review_events`
Eligibility scoring	FR-12	`eligibility.controller.js`, `services/eligibility.js`	`pages/Eligibility.jsx`	`students.eligibility_*`
Bursary awards	FR-13 to FR-15	`bursaryAwards.controller.js`	`pages/Disbursements.jsx`	`bursary_awards`, `bursary_award_verification_events`
Payments and reconciliation	FR-16 to FR-19	`disbursements.controller.js`, `bursaryRules.js`, `config/reconciliation.js`	`pages/Disbursements.jsx`	`disbursements`, `disbursement_reconciliation_events`
Student QR verification	FR-20, FR-21	`verification.controller.js`, `verificationRateLimit.js`, `publicAppUrl.js`	`pages/Verification.jsx`, `hooks/useHashToken.js`	`students.qr_token`
Award certificates	FR-22 to FR-24	`certificates.controller.js`, `bursaryCertificate.model.js`, `certificateDocument.js`, `certificateRateLimit.js`	`components/AwardCertificatePanel.jsx`, `pages/CertificateVerification.jsx`	`bursary_certificates`
Dashboard and reporting	FR-25, FR-26	`adminDashboard.controller.js`, `reports.controller.js`	`pages/AdminDashboard.jsx`, `pages/Reports.jsx`	read-only queries, `reports`
Audit and administration	FR-27 to FR-29	`audit.controller.js`, `auditLog.js`, `users.controller.js`, `settings.controller.js`	`pages/AuditLogs.jsx`, `pages/UserManagement.jsx`, `pages/Settings.jsx`	`audit_logs`, `users`
Operations	FR-30	`databaseReadiness.js`, `databaseStartup.js`, `scripts/migrate.js`, `scripts/bootstrap-production-schema.js`	—	`sbms_schema_migrations`

5.3.2 Authentication and access control

The sign-in path is implemented as one controller function parameterised by the expected role, which is why the two entry points cannot drift apart. The sequence is: validate the body with Zod (e-mail format, non-empty password) → fetch the account by lower-cased e-mail → reject when the account is missing, inactive, or the bcrypt comparison against the stored hash fails, in all three cases with the same `401 Invalid email or password` so that the response does not reveal which accounts exist → compare the account role with the role the endpoint expects and answer `403` with an instruction naming the other form → sign `{id, role}` with an eight-hour expiry and return the token with the profile (`controllers/auth.controller.js:37-74`). The recorded state of the disposable test database after a full suite run was `users = 4`, created by the suites themselves; the deployed database holds the administrator and field-officer accounts whose passwords are kept only in the git-ignored `deploy/azure/.secrets.json`.

The authorisation path is deliberately database-backed: `authenticateToken` verifies the signature, then reads `id, role, is_active` from `users` and rejects the request if the row is gone or inactive, replacing whatever role the token claims with the stored role (`middleware/auth.js:17-25`). The consequence is testable and was tested: a token signed with `role: 'admin'` for a field-officer account still yields 403 on admin routes, and a role change takes effect for tokens already issued (`tests/roleAuthorization.test.js`, "signed client role claims cannot grant admin access and role changes apply to existing tokens").

Table 5.4: Authentication and access-control implementation details

Behaviour	Implementation	Verified by
Password hashing	`bcrypt.hash(password, 12)` on creation; `bcrypt.compare` on sign-in	`controllers/auth.controller.js:26, 48`; `tests/auth.test.js`
Token lifetime and claims	`jwt.sign({id, role}, secret, {expiresIn:'8h'})`	`controllers/auth.controller.js:55-59`
Secret policy	Missing, shorter than 32 bytes or placeholder secrets block start-up	`config/jwtSecret.js`; `test/jwtSecret.validation.node.cjs` (part of the 43 native tests)
Public registration	Always `410 Public registration is disabled.`	`routes/auth.routes.js:9-11`; `tests/auth.test.js`
Wrong-role sign-in	`403` with a message naming the other sign-in form	`controllers/auth.controller.js:51-53`; `tests/roleAuthorization.test.js`
Sign-in throttling	10 attempts per IP per 15 minutes, shared counter for both endpoints	`middleware/loginRateLimit.js`; `tests/loginRateLimit.test.js`
Token required	`401` when the `Authorization` header is absent or not a `Bearer` token	`middleware/auth.js:6-12`; `tests/students.test.js`

5.3.3 Student and beneficiary management

Registration is the most involved controller in the system because it coordinates validation, token generation, model scoring, flag aggregation, persistence and audit inside one transaction. It is implemented in `controllers/students.controller.js:31-161` in an order that makes failure safe:

1. **Validation** by Zod: name trimmed and whitespace-collapsed to 2–255 characters; date of birth checked as a real ISO date; `school_code` restricted to the five approved schools; identifiers normalised to upper-case alphanumerics and capped at 100 characters; phone normalised to `+254[17]XXXXXXXX`. A failure returns `400 {message:'Invalid student data', errors}` and nothing is written.
2. **Policy resolution before the transaction**: the review threshold and the aggregation policy are read and validated, so an invalid `DUPLICATE_AGGREGATION_POLICY` fails the request without opening a connection (`students.controller.js:54-57`).
3. **QR token and verification URL**: 32 random bytes as 64 hex characters, and the URL built from the validated `PUBLIC_APP_URL`; a misconfigured origin fails here rather than creating a student whose code cannot be scanned.

4. **Transaction and lock**: `BEGIN`, then `pg_advisory_xact_lock(hashtext('sbms.student.registration'))`, then every existing student selected `FOR UPDATE`. The lock serialises registration against award approval and payment creation, which use the same advisory key.
5. **Scoring** of candidate pairs in batches of 200.
6. **Failure path**: if the predictor is unavailable, times out or returns a malformed payload, the transaction rolls back and the client receives `503`. **No student is created** — an unscreened registration is treated as worse than a failed one.
7. **Selection and persistence**: qualifiers are reduced by the aggregation policy, the student row is inserted, and one `duplicate_flags` row is written per selected candidate with score, model version, threshold and feature vector. The unordered-pair unique index turns a repeat creation into a no-op rather than a duplicate case.
8. **Audit and response**: an audit row records the policy, the three candidate counters and the number of flags created; the response returns the student **without** `qr_token`, the QR image, the flags and the screening summary.

Record management beyond creation is deliberately narrow: `GET /api/students` lists the registry, `GET /api/students/:id` reads one record, `GET /api/students/schools` returns the approved list, and there is **no update or delete endpoint**, so a beneficiary record cannot be silently edited or removed through the API. Eligibility assessment is the only operation that mutates a student row after creation (§5.3.6).

Table 5.5: Validation rules implemented on the student record

Field	Rule	Enforced by
`full_name`	2–255 characters, internal whitespace collapsed	Zod schema
`date_of_birth`	Optional, but if present a real `YYYY-MM-DD` calendar date	`validIsoDate` in Zod
`school_code`	One of `Nairobi School`, `Alliance High`, `Bishop Gatimu`, `Loreto Limuru`, `Maranda`	Zod enum **and** `students_school_supported_check`
`guardian_id`, `upi`, `birth_certificate_number`, `national_id_number`	Optional, ≤ 100 characters, normalised to alphanumerics, upper-cased	`normalizeIdentifier`
`primary_phone`	Optional, `+254[17]XXXXXXXX` after normalisation; `0712345678` and `+254712345678` both accepted	`normalizeKenyanPhone`
`qr_token`	Server-generated only; a client-supplied value cannot enter because the schema does not accept it	`studentSchema` (token excluded by design)

5.3.4 Machine-learning-assisted duplicate detection

Production artefact and feature schema. The application loads exactly one artefact, `ml-service/model/xgb_duplicate_model.joblib` — 421 089 bytes, SHA-256 `6ec9f2a3cbe6ed0dcf80c79b15f47364d9f4f7bd3e27da25c690d0436bb0a6f9` — a scikit-learn pipeline containing an imputer and an `XGBClassifier`. The version reported to reviewers is `xgboost:<sha256>`, computed by reading the artefact at process start (`predict.py:68-70`), so a replaced artefact cannot masquerade as the evaluated one. The six features and their order are fixed in one list used by training, evaluation and serving (`feature_engineering.py:11-18`), and the training report asserts that the serving path reproduced the generator's stored feature columns exactly (`feature_consistency.exact_match = true`).

How a registration reaches the scorer. For every existing student the controller builds the pair `{first: submitted, second: candidate}` and calls `scorePairs` (`students.controller.js:69-73`). `scorePairs` normalises `date_of_birth` to a calendar date at this boundary, chunks the list into 200-pair batches, writes one JSON line per batch to the predictor's standard input, and validates the reply: result count must equal pair count, every score must be finite and within [0, 1], every result must carry a feature object, and `model_version` must not change between batches (`mlDuplicateReview.js:84-117`). The predictor loads the artefact once, builds features with the shared module, calls `predict_proba` and returns `{score, features}` per pair (`predict.py:40-65, 84-98`).

The role of Jaro–Winkler and the other features. `jaro_winkler` is the strongest name-level signal and is computed twice — on concatenated tokens and on alphabetically sorted tokens — with the higher value kept, so "Wanjiku Mary" and "Mary Wanjiku" score highly rather than poorly. `dob_match` is banded (1.0 exact, 0.8 one day, 0.5 up to a week) because typists transpose day and month. The identifier and school features express agreement rather than similarity of meaning: school agreement is binary, while guardian identifier, UPI and birth-certificate number compare exactly after stripping shared prefixes (such as `G`, `UPI`, `BC`) and fall back to Jaro–Winkler for partial agreement, so a mistyped digit yields a partial score rather than zero.

How the score becomes a case. A candidate *qualifies* if its score reaches the review cut-off **or** if UPI, birth-certificate number or national identifier agrees exactly — a signal the model does not see. Qualifiers are ranked with exact identifier agreement above score, and the aggregation policy decides how many become cases (default: one). The case stores the score as `probability`, the artefact hash, the threshold and the feature vector, and the response reports how many candidates were considered, qualified and suppressed (`duplicateAggregation.js:119-144`).

How the human decides. The administrator sees both records, the priority band, the score, the model version and the threshold, and records `confirmed_duplicate`, `not_duplicate` or `needs_verification` with a reason. The decision writes a `duplicate_review_events` row and an `audit_logs` row carrying the model version and score (`duplicateFlags.controller.js:452-503`). Nothing converts a score into a rejection: there is no delete, no status change and no block on registration.

Separation from research artefacts. Three artefacts exist; one is served. The production model is the six-feature XGBoost artefact above. `ml-service/model/research/rf_attachment_style_candidate.joblib` is a Random Forest candidate trained on attachment-style pairs, with grouped cross-validation F1 0.700–0.732, holdout F1 0.695 and recall 0.538, and is never loaded by the application (`ML_RELIABILITY_REPORT.md`). `ml-service/experiments/sbms_dataset/artifacts/sbms_dataset_xgb_research_only.joblib` is a ten-column model trained on a supplied external synthetic dataset whose schema has no UPI or birth-certificate fields and whose school labels do not correspond to the five approved schools, so it is explicitly incompatible with the serving contract (`ml_audit.md`; `reports/sbms_dataset_experiment.md`). Its test F1 of 0.875 and PR-AUC of 0.953 belong to that experiment and are not properties of the production system.

5.3.5 Duplicate review and resolution

The review queue is a joined query returning, for each case, both students' names, dates of birth and school identifiers, with `guardian_id`, `upi`, `birth_certificate_number` and `national_id_number` masked to their last three characters by `maskIdentifier` (`duplicateFlags.controller.js:37-42`). The queue also returns a policy block: the review threshold, the high-priority threshold, the aggregation policy and `thresholds_validated: false`. That flag is not decoration — it is how the interface states that the cut-offs are provisional.

Three operations complete the module. `evaluate` scores supplied pairs without persisting anything, which is what makes the model testable through the same HTTP surface as production. `scanExisting` re-scores the existing registry, enumerating pairs and skipping those already flagged, and is quadratic in registry size. `revealIdentifiers` returns the five identifier fields for both records of one case and writes a `REVEAL_DUPLICATE_IDENTIFIERS` audit row naming the fields disclosed, the two student identifiers and the stated purpose (`duplicateFlags.controller.js:434-446`).

The decision is transactional and terminal: the `UPDATE` is constrained to `status IN ('pending','needs_verification')`, so a case is decided once and a second attempt receives 404; the review event and the audit row are written in the same transaction; and `resolved_at` is set only for the two terminal decisions, leaving `needs_verification` open with a recorded reason.

5.3.6 Eligibility scoring

Eligibility is a transparent rule engine rather than a second model, as the proposal justified (§1.4). `services/eligibility.js` awards points across five factors and returns the total, a category, a per-factor breakdown and human-readable reasons. The controller validates that the three numeric inputs are finite and non-negative and

that the two categorical inputs are in their enumerations, then optionally writes the score and category to the student row.

Table 5.6: Eligibility scoring rules as implemented

Factor	Points	Thresholds
Fee balance	30 / 25 / 15 / 5 / 0	≥ 50 000 / ≥ 30 000 / ≥ 15 000 / > 0 / 0
Household income	25 / 20 / 10 / 0	≤ 20 000 / ≤ 40 000 / ≤ 60 000 / > 60 000
Vulnerability status	25 / 15 / 5	high / medium / low
Siblings in school	10 / 7 / 3 / 0	≥ 4 / ≥ 2 / = 1 / 0
Prior bursary history	10 / 7 / 2	none / low / high
Category	—	≥ 75 `highly_eligible`; ≥ 50 `eligible`; ≥ 30 `partially_eligible`; otherwise `not_eligible`

The dashboard counts `eligible` and `highly_eligible` students, which is the only place the score is used outside the assessment screen. **No automated test of the scoring arithmetic exists**: the route and its role restriction are covered, and the values above were verified by reading `services/eligibility.js` against `docs/eligibility-rules.md`, which agree.

5.3.7 Bursary awards, payments and reconciliation

The award and payment module is the system's answer to the documentation failures described in Chapter One, implemented as a chain of explicit gates rather than as a status field.

Award submission requires an existing student with a school on record and creates a row in `submitted` state with the requested amount (`bursaryAwards.controller.js:64-95`). **Verification** is recorded by the field officer as `verified`, `not_found` or `needs_investigation`, with a mandatory note and — for `verified` — a mandatory evidence reference, and each recording appends an event row so the history of verification attempts is preserved rather than overwritten.

Approval is the first gate. Inside a transaction that takes the same advisory lock as registration, the controller refuses approval when the award is not `submitted` (409), when the beneficiary is not `verified` (409), or when any duplicate case for that student is `pending`, `needs_verification` or `confirmed_duplicate` (409) — and the duplicate query tests **both** `student_id` and `matched_student_id`, so a student is blocked whether they are the new record or the existing record they were matched against (`bursaryAwards.controller.js:173-189`). Only then are status, approved amount, reason, deciding user and time written, with an `AWARD_APPROVED` audit row.

Payment is the second, independent gate. The controller refuses a payment when the award is not `approved`, when verification is no longer `verified`, when a duplicate case exists, or when the sum of existing non-failed, non-cancelled payments plus the new amount would exceed the approved amount (`disbursements.controller.js`). A payment is recorded as `issued` — never as paid — with `receipt_status = 'awaiting_acknowledgement'`, and appends a `payment_issued` reconciliation event.

Settlement is recorded separately. `updatePaymentStatus` accepts only `issued → cleared`, `issued → failed` and `issued → cancelled` (`bursaryRules.js:1-13`); clearance requires an evidence reference, and failure or cancellation requires a reason of at least five characters. A receipt acknowledgement compares the payment and acknowledged amounts in integer cents and stores `acknowledged` when they agree and `discrepancy` when they do not, so a shortfall is preserved as evidence rather than corrected (`bursaryRules.js:15-22`).

Exceptions are derived rather than stored: `exceptionReasons` evaluates each payment against eleven conditions — legacy payment without an award link, unverified payment status, incomplete beneficiary verification, amount above the approved award, partial payment, no payment recorded, awaiting clearance, failed, cancelled, awaiting

acknowledgement, acknowledgement overdue (when `ACKNOWLEDGEMENT_OVERDUE_DAYS` is set) and amount discrepancy (`bursaryRules.js:32-52`). Because they are computed at read time they cannot drift from the underlying data.

Two boundaries must be stated because a reader will otherwise over-read this module. First, the system does **not** transfer money: it records that a payment was issued and, separately and with evidence, that it cleared. Second, receipt acknowledgement is **officer-assisted** — the database requires `acknowledgement_method = 'officer_assisted'` — so it is not independent confirmation by a school, and the interface and the project records both say so.

5.3.8 Student QR verification

The QR token is generated during registration: `randomBytes(32).toString('hex')`, 64 lowercase hexadecimal characters, stored in `students.qr_token` under a partial unique index and encoded into a QR image whose payload is `PUBLIC_APP_URL + '/verify#token=<token>'`. The token sits in the **fragment**, so it is never sent to a web server, never written to an access log and never forwarded as a `Referer` (`config/publicAppUrl.js:164-168`). Registration and list responses strip `qr_token` (`models/student.model.js:3-7`), so the token reaches the officer only inside the QR image.

The public page reads the fragment, removes it with `history.replaceState`, and posts `{token}` to `/api/verify`. Because a hash-only change does not remount a React component, the page re-runs verification on `hashchange` — a defect found in browser testing and fixed (`docs/qr-verification-2026-10-30.md` §4). The endpoint requires exactly one key and the 64-hex shape, answers `{found:true}` or `{found:false}` and nothing else, returns the same answer for malformed and unknown tokens, and returns `503 {error:'temporarily_unavailable'}` on a lookup failure. Its rate limiter runs in the same route chain ahead of the JSON parser, so an abusive client is refused before its body is read.

The information revealed is deliberately exhausted by that boolean. It does not confirm identity, eligibility, award approval or payment, and the page states so in its result text. There is **no token expiry, rotation or re-issue path**: a token is valid for the lifetime of the student row, and a row created without a token cannot be given one through the API (`docs/qr-verification-2026-10-30.md` §6).

5.3.9 Bursary award certificates

A certificate is issued only for an approved award, and the issue path refuses an award that already has a certificate (`certificates.controller.js:95-181`). Issuing writes a **snapshot row**: reference `SBMS-CERT-<year>-<8 uppercase hex>`, a fresh 64-hex verification token, the award period (defaulting to the UTC year of approval), the student's name, UPI and school, the approved amount at issue, and the approval timestamp. The snapshot exists so that a later change to the award cannot rewrite what a printed document says. The response returns the certificate facts, the verification URL and a QR data URL; the printable PDF is rendered per request with `pdfkit` and `qrcode` on A4 landscape and served with `Cache-Control: no-store`.

Public verification answers in one of four states, computed by comparing the snapshot with the live award (`certificates.controller.js:53-89`): `verified` when the certificate is active and the award is still `approved` at the same amount; `not_current` when the award is no longer approved or its approved amount has changed; `revoked`; and `not_recognised` for any malformed, unknown or manipulated token. A `verified` response discloses the certificate reference, award period, issue date, approval date, award status and the issuing authority, and **not** the student's name, UPI, school or the amount. Revocation is a status change with actor, timestamp and a reason of at least five characters, enforced by a table `CHECK` as well as by the API, and it never alters the award or any payment.

The distinction between the two verification functions is enforced technically, not only conceptually: the endpoints have separate paths, separate token namespaces, separate rate-limit budgets and separate pages, and a certificate token does not verify a student record or the reverse (`tests/bursaryCertificates.test.js`).

5.3.10 Dashboard, reporting and audit

The dashboard is six parallel queries executed with `Promise.all`: beneficiary counts (total, eligible plus highly eligible, synthetic), open duplicate cases, a six-month registration trend, a six-month disbursement trend, the four

most recent audit actions, and an accountability block of nine counters (approved awards, disbursed awards, approved awards with no payment, disbursed amount, awaiting clearance, awaiting receipt, overdue receipts, partial awards and amount discrepancies). The trends use a `generate_series` CTE with a window `SUM`, so a month with no activity appears as a flat point rather than disappearing. The response also carries `acknowledgementOverdueDays` and `lastUpdated`.

Reporting is one endpoint that requires both dates, refuses a reversed range, queries the same accountability snapshot, writes a `reports` row and a `GENERATE_REPORT` audit row, and streams a `pdfkit` document containing a summary, an accountability snapshot and a disbursement detail list. It is an administrator-only, on-demand export; there is no scheduled report generation.

Auditing is a single function, `logAudit(userId, action, tableName, recordId, details, executor)`, called with the transaction client wherever the operation is transactional, so an audit row cannot be committed without its business change or vice versa. The action vocabulary includes `CREATE`, `REVEAL_DUPLICATE_IDENTIFIERS`, `DUPLICATE_CONFIRMED_DUPLICATE`, `DUPLICATE_NOT_DUPLICATE`, `DUPLICATE_NEEDS_VERIFICATION`, `AWARD_VERIFICATION_RECORDED`, `AWARD_APPROVED`, `AWARD_REJECTED`, `PAYMENT_ISSUED`, `PAYMENT_CLEARED`, `RECEIPT_ACKNOWLEDGED_OFFICER_ASSISTED`, `LEGACY_PAYMENT_LINKED_TO_AWARD`, `AWARD_CERTIFICATE_ISSUED`, `AWARD_CERTIFICATE_REVOKED`, `ADMIN_UPDATE_DUPLICATE_THRESHOLDS` and `GENERATE_REPORT`.

5.3.11 Configuration, start-up guards and migration control

Start-up is deliberately strict, because the system's deployment failure modes were configuration failures rather than code faults. Before binding a port the API validates the JWT secret (present, at least 32 bytes, not a known placeholder), resolves `PUBLIC_APP_URL` through the public-origin validator, and asserts that the expected database name matches and that `users` carries the columns the login query needs. Any failure writes one diagnostic line naming the guard and exits with status 1, so a container cannot enter a crash loop silently. `GET /api/health` reports liveness; `GET /api/ready` runs the readiness contract against the required tables and the expected database identity and returns `{"status":"ready"}` or a generic `503`. Neither endpoint creates or migrates anything.

Configuration is read from the environment: `PORT`, `DATABASE_URL`, `SBMS_EXPECTED_DATABASE_NAME`, `JWT_SECRET`, `CORS_ORIGINS`, `PUBLIC_APP_URL`, `PYTHON_EXECUTABLE`, `TRUST_PROXY_HOPS`, `ML_REVIEW_SCORE_THRESHOLD`, `DUPLICATE_REVIEW_THRESHOLD`, `DUPLICATE_HIGH_PRIORITY_THRESHOLD`, `DUPLICATE_AGGREGATION_POLICY` and the optional `ACKNOWLEDGEMENT_OVERDUE_DAYS`. Values are never printed; the README documents how to generate a secret and warns against reusing the production secret in tests.

One configuration behaviour deserves emphasis because it is easy to misread as a defect. Threshold changes made through `PUT /api/settings/threshold` are held in **process memory** and audited, but not persisted: a restart restores the environment defaults. The screen and the audit record do not claim otherwise, but an operator must know it.

5.3.12 Deployment implementation

Deployment is a single container image built from the repository `Dockerfile`, published to `ghcr.io/eggxellent/sbms-api` by a manually dispatched GitHub Actions workflow and run by Azure Container Apps. The build is not a plain `docker build`: it installs Python and creates a virtual environment, installs `xgboost` with `--no-deps` so that a ~350 MB CUDA dependency is never pulled, then asserts that no `nvidia-*` package is present; copies only `predict.py`, `feature_engineering.py` and the single production artefact; and runs an inference gate that loads the artefact, scores a known-duplicate pair above 0.9 and a known-unrelated pair below 0.5, and asserts the exact six-feature order. The workflow repeats the model gate and a secrets gate against the pushed image. An image that cannot score therefore cannot be published.

The frontend is built with `VITE_API_ORIGIN` pointing at the API origin, zipped and published to App Service, where a `web.config` rewrite lets client routes resolve while deliberately leaving `/api/` alone.

Table 5.7: Deployment checks recorded in the live deployment log

Check	Recorded result
Start-up script	PostgreSQL `Ready`; `/api/health` 200; `/api/ready` 200; frontend 200 with `#root`
Field-officer sign-in through the deployed form	200; `/field-officer` renders the officer dashboard
Administrator sign-in	200; `/dashboard` renders the admin dashboard
Admin surfaces	`/api/admin/dashboard`, `/api/duplicate-flags`, `/api/awards`, `/api/disbursements`, `/api/users`, `/api/audit-logs` → 200; `/api/students` → 403 (intended)
Student QR scan through the deployed SPA	Initially **broken** (`POST https://sbms-web.azurewebsites.net/api/verify` → 404) because the page used a relative `fetch`; after routing the call through the axios client, headless-Chrome `qr-smoke` passed **9/9** against the deployed site
Real-browser sign-in for both roles	**9/9**
Officer screens	`/students` rendered the live records (3.2 s warm, measured from `performance.getEntriesByType('resource')`); `/students/register` rendered its inputs
Live screening with shared identifiers	A second student sharing UPI and birth-certificate number produced `exact_identifier_matches: ["UPI","birth certificate number"]`, `birth_certificate_match: 1`, `upi_match: 1`, `model_score: 0.9382`, `priority: high`
Registration-fields smoke against production	**11/11**

The same log records what remains untidy: two probe students created during the live check were still in the production database at the time of that record; the stored administrator password had drifted from the operator secret file, so the admin reveal screen was not re-checked; and the database was left running, with a re-stop due by 8 November 2026 (`docs/azure-deployment-log-2026-10-31.md` §7–§9).

5.4 System testing

5.4.1 Testing strategy and levels

Testing was organised in five levels, each answering a different question, and the levels are reported separately because they carry different weight.

Table 5.8: Testing levels and what each establishes

Level	Instrument	Question answered	Scope in this project
Unit	Jest, `node --test`, Python `unittest`	Does a function or module behave as specified in isolation?	Feature engineering, aggregation policy, priority rules, date normalisation, payment state machine, error sanitisation, JWT validation
Integration (API + database)	Jest + Supertest against a real PostgreSQL database	Do routing, middleware, controllers, services and SQL cohere through the real HTTP surface?	24 suites, 151 tests, including an end-to-end business-workflow suite
Out-of-process smoke	`node --test backend/test/api.smoke.test.js` against a running API	Does a server started the way an operator starts it answer correctly?	3 checks: health, admin authentication with role boundary, report range validation
Browser (UI)	Headless Chrome over the DevTools Protocol	Does the rendered application perform the workflow a human performs?	23 registration/review checks, 9 QR checks, 11 registration-field checks against the deployed site
Deployment	Manual and scripted checks against Azure	Does the deployed configuration behave like the tested configuration?	Health, readiness, both sign-ins, admin surfaces, QR scan through the SPA, live screening

5.4.2 Test environment and provenance of results

Two environments carry the results, and the reader must know which produced what.

Disposable database. Database-backed suites run against `sbms_test` on loopback with role `sbms_test_runner`, under a guard that **fails closed** unless `NODE_ENV=test` and both `DATABASE_URL` and `SBMS_TEST_DATABASE_URL` name that same loopback database (`backend/tests/testDatabaseGuard.js`). The guard makes it impossible for these runs to have touched a production database, and it is why the suites abort rather than skip when the environment is wrong. The recorded state of that database after a full run was 12 base tables, 118 columns, 56 constraints, 21 indexes and 12 applied migrations, containing `users = 4`, `audit_logs = 1` and **zero** students, flags, awards and payments, which also shows that the suites clean up after themselves.

Provenance. The figures below are taken from the project's own records, which name the command, the date and the commit: `docs/FINAL_TEST_REPORT.md` (§1–§10, covering 2026-10-31 and 2026-11-01), `docs/business-workflow-verification-2026-10-30.md`, `docs/frontend-verification-2026-10-30.md`, `docs/qr-verification-2026-10-30.md` and `docs/bursary-certificates-2026-11-01.md`. Those runs were **not re-executed** while this chapter was written; the suites and their assertions were re-read to confirm that the reported counts and behaviours correspond to the code now in the repository. Where records disagree, the newest is cited and the older is identified.

5.4.3 Automated backend suite: test-case inventory

Table 5.9 (part 1 of 2): backend Jest suites and what each establishes

#	Suite	Tests	What it establishes
1	`auth.test.js`	4	Public registration returns 410; field-officer sign-in issues a JWT; invalid tokens and wrong-role access are refused
2	`students.test.js`	9	Authentication required; registry listing; five-school list; unapproved school rejected; admin refused; birth-certificate persistence; reused UPI queues a case; exact birth-certificate agreement prioritised
3	`duplicateFlags.test.js`	9	Review threshold exposed distinctly; queue returns pending cases; reveal audited; paired scoring without persistence; non-admin refused; decision recorded in case history; confirmed duplicate blocks disbursement; pending and needs-verification block until resolved
4	`duplicateAggregation.test.js`	13	`top-1` default; policy read and validated; one flag from many qualifiers; exact identifier above score; deterministic tie-break; below-cutoff candidate flagged only with an exact identifier; `all` retains legacy behaviour; corroboration suppression and its exception; counters; corroboration ignores name and school; missing threshold rejected
5	`mlDuplicateReview.test.js`	8	Batched scoring in order; version reported with no pairs; national-ID exact match as a signal; UPI and birth-certificate signals; exact agreement outranks score; score-only cases use the high-priority cutoff; fallback cutoff; no throw on malformed input
6	`predictorDateNormalization.test.js`	6	`pg` DATE serialised as its calendar date; normalised values unchanged; ISO timestamp truncated; missing values null; unrecognised text passed through; only the date field mutated
7	`duplicateCheck.test.js`	15	Behaviour of the **legacy** rule engine, retained for comparison; not a production path
8	`mlPredictor.integration.test.js`	1	The deployed artefact scores a synthetic pair and reports its version
9	`workflow.test.js`	11	The whole chain: case creation, verification gate, duplicate gates on approval **and** payment, terminal confirmed duplicate, non-re-decision, `needs_verification` clearing, approved-amount ceiling, clearance evidence and reconciliation trail, acknowledgement discrepancy, ten-action audit trail, role boundaries
10	`bursaryAwards.controller.test.js`	5	Approval of a verified award with no open case; rejection with reason; blocked before verification or review; malformed decision refused before a transaction; route guards
11	`disbursements.test.js`	4	Authentication required; invalid amount refused; `issued` payment for an approved verified award; payment blocked while a case is pending
12	`disbursementReconciliation.controller.test.js`	4	Split payments only within the approved balance; clearance requires evidence and is audited; acknowledgement mismatch recorded as an exception; clearing without evidence refused before a transaction

Table 5.9 (part 2 of 2): backend Jest suites and what each establishes

#	Suite	Tests	What it establishes
13	`bursaryRules.test.js`	4	Payment transition legality, cent arithmetic, exception derivation, overdue computation
14	`bursaryCertificates.test.js`	12	Approval as precondition; admin-only issue and revoke; malformed period refused; QR payload is the verification URL; no second certificate; staff read and PDF download; public authenticity without beneficiary data; identical refusal for unknown, malformed and manipulated tokens; token namespaces kept apart; `not_current` after an amount change; revocation leaves the award untouched; audit rows
15	`certificateVerificationRoute.test.js`	5	Handler reached only through the limiter; limiter before the parser; size limit still enforced; only POST routed; separate student and certificate budgets
16	`verification.test.js`	10	64-hex token; QR encodes the fragment URL; token never returned; valid token yields only `found:true`; unknown and malformed tokens indistinguishable; extra fields and empty body refused; budget stops the eleventh request; forged `X-Forwarded-For` does not reset the budget
17	`verificationRoute.test.js`	4	Single registration with the limiter before the parser; size limit enforced; only POST routed; response exposes nothing but `found`
18	`roleAuthorization.test.js`	6	Unauthenticated callers refused; admin lists and creates accounts; separate sign-in endpoints; synthetic counts and admin-only dashboard; student routes stay field-officer-only; field officers cannot manage users or thresholds; forged role claims and role changes take effect
19	`securityMiddleware.test.js`	9	Development default and production fail-closed; origin list parsing; security headers; trusted and untrusted preflight; pre-routing origin rejection; oversized JSON; malformed JSON; no message or stack leakage; limiter on both sign-in routes
20	`schemaReadiness.test.js`	5	Readiness contract lists every required table; built schema reported ready; the legacy table has the columns the model uses; the mounted beneficiaries query executes; a row round-trips
21	`loginRateLimit.test.js`	1	Ten sign-in attempts allowed, the eleventh refused
22	`performance.test.js`	1	A 50-candidate registration is screened within three seconds
23	`kenyanSyntheticDataset.test.js`	4	50 marked records and 21 labelled pairs; typo variants are candidates and reversed order is not assumed exact; shared school and guardian alone do not make a different-age sibling a candidate; missing values are zero evidence
24	`health.test.js`	1	Liveness endpoint returns 200

Recorded result, newest full run (2026-11-01): `24 suites passed, 151 tests passed`, exit code 0 (`docs/FINAL_TEST_REPORT.md` §10; `docs/bursary-certificates-2026-11-01.md` §7). An earlier run on the same branch (2026-10-31, before the certificate feature) recorded `21 suites / 128 tests` with wall-clock times of 93.6 s and 73.5 s; it is retained because it shows the count depends on the revision, not the machine.

An honest note on stability. During the documentation work that preceded this chapter, three repeated `--runInBand` sweeps each completed 149 or 150 of 151 tests, with the failing suite varying between runs and always being one that reaches the Python predictor (`duplicateFlags`, `mlPredictor.integration`, `students`). The failure surfaced as `Python predictor process exited` → HTTP 503 `DUPLICATE_SCREENING_UNAVAILABLE`, and each of those suites passed alone (`duplicateFlags` 9/9, `duplicateCheck` + `mlDuplicateReview` 23/23, `mlPredictor.integration` 1/1). The cause identified was host memory pressure — about 0.5 GB free of 7.4 GB — combined with the design fact that `duplicateCheck.js` keeps an unreferenced predictor child process, with XGBoost and pandas resident, alive for the lifetime of each test file, so several accumulate during a sequential sweep and the operating system terminates one. This is an environment-dependent instability, not a code defect, and it is also a genuine operational limitation of the single-process predictor design (§5.6.5). The green 151/151 sweep in the project report remains the authoritative result for the revision.

5.4.4 Native, smoke and Python testing

Native in-process suite — 43/43. Seven files run under `node --test` without a live database: `databaseReadiness.node.js`, `databaseStartup.node.test.js`, `databaseBootstrap.node.test.js`, `errorHandler.unit.test.js`, `verification.unit.test.js`, `migrate.unit.test.js` and `jwtSecret.validation.node.cjs`. Recorded: `# tests 43 # pass 43 # fail 0`, 0.79 s, exit 0. Coverage includes the readiness contract (missing or shadowed table, wrong database, unset expected name, malformed catalogue rows, error propagation), start-up validation, the fail-closed empty-schema bootstrap (approval gate, wrong database, pre-existing relation, atomic rollback, bcrypt admin hash, e-mail collision), error-handler sanitisation, JWT-secret validation, migration target, inventory and ledger adoption, and QR verification including the rate-limit boundary.

Out-of-process API smoke — 3/3. `backend/test/api.smoke.test.js` fetches a live API; run in-process it fails on its prerequisites (`fetch failed`), which is expected and documented. Run against a server started the operator's way on `sbms_test`, it recorded `# tests 3 # pass 3 # fail 0`, exit 0, covering health, admin authentication with the field-officer boundary, and report range validation, with `/api/ready` returning `{"status":"ready"}`.

Python ML tests — 33/33 and 7/7. `python -m unittest discover -s ml-service/tests` recorded 33 passing tests in 1.96 s, covering feature engineering (shared training/serving features, missing-value handling), operating-point and threshold metrics, registration simulation, artefact loading (only the production artefact; missing or corrupt artefacts fail closed), training-integrity reporting (group holdout excludes pair families, the model does not memorise training rows, no single feature determines the label) and reliability analysis. `python -m unittest discover -s ml-service/experiments/sbms_dataset -p test_experiment.py` recorded 7 passing tests in 0.47 s. `pytest` was **not** run because it is not installed in that environment (`importlib.util.find_spec('pytest') -> None`), and no package was installed to change that.

5.4.5 Database, security and performance testing

Database integration. The Jest suites run against PostgreSQL 17.11, not mocks, and are gated so they cannot touch another database. Their value is not only that they exercise SQL but that they exercise the constraints: the workflow suite proves that a database `CHECK` refuses an approval without verification, a clearing event without evidence and an acknowledgement without a method, and the certificate suite proves that the revocation and token-format `CHECK`s behave. Database recovery was separately rehearsed end to end — custom-format dump, restore into an independent database, readiness check, schema and row-digest comparison, and an application smoke test against the restored copy (`docs/database-recovery-rehearsal-2026-10-31.md`). No backup schedule was configured, so this is a rehearsed procedure rather than an operating control.

Security testing. `securityMiddleware.test.js` (9) and `roleAuthorization.test.js` (6) carry the security evidence: origin allowlist enforced before routing, trusted and untrusted preflight, security headers, oversized and malformed JSON handled without leakage, no exception message or stack returned, both sign-in routes limited, and the role matrix enforced including the "forged claim" case. Rate limiting is verified at its boundary in three suites, including the check that a forged `X-Forwarded-For` header does not create a fresh budget.

Performance. `performance.test.js` records that a 50-candidate registration is screened within the three-second requirement, and the 2026-10-30 record measured 22.67 ms for 50 pairs with a warm predictor. This is a single-point measurement at a small registry size. There is **no** measurement at 500 or 1 000 candidates, no hosted load test and no concurrency benchmark, so the requirement is verified only for the case tested, and behaviour under concurrent registration is asserted by the advisory lock rather than measured. Screening time also grows with registry size, because every registration is compared against every student (§5.6.5).

5.4.6 Browser-based (user-interface) testing

Three browser harnesses drive real Chrome through the DevTools Protocol, signing in through the real forms and reading the rendered DOM.

Table 5.10: Browser test results by harness

Harness	Checks	Recorded result	What it establishes
`scripts/browser-smoke.mjs`	23	**23/23 pass** (2026-10-30)	Field-officer sign-in; normal registration; screening sentence shown; no model internals to the officer; re-registration created rather than rejected; duplicate warning; exact identifier named; red styling for a suspicious case; one case from several qualifying candidates; blocked API surfaces an error and no confirmation panel; admin sign-in; queue shows the case as pending; both cutoffs and the aggregation policy visible; model version and cutoff in case detail; score labelled as not a decision; audit counters rendered readably; admin redirected away from the registration page
`scripts/qr-smoke.mjs`	9	**9/9** locally (2026-10-30) and **9/9** against the deployed site after the fix (2026-11-01)	Valid code opens the page and reports the record; token removed from the visible URL; no student data displayed; unknown token refused; missing token refused; query-string token not accepted; second scan in the same tab re-verifies; no uncaught page errors
`scripts/registration-fields-smoke.mjs`	11	**11/11** against the deployed site (2026-11-01)	Both optional identifier inputs present and not required; the six original fields still posted; the payload carries both markers; a blank identifier omitted; a blocked request shows the error panel

Two defects were found at this level and would not have been found by unit tests: the stale-result behaviour of the verification page across two scans in one tab, and — in production — the relative `fetch("/api/verify")` that made every scan fail on the App Service host while the same endpoint answered correctly when called directly on the container app.

5.4.7 Defects found through testing and their resolution

Testing was not only confirmatory. The following defects were found by executing the system, and each is included because it is evidence about the system's quality rather than against it.

Table 5.11: Defects found and fixed

#	Defect	How it was found	Root cause	Resolution
D-01	Payment clearance returned 500 for every status change, so reconciliation was unreachable	The new end-to-end workflow suite exercised the endpoint for the first time	PostgreSQL `42P08`: one parameter reused in three type positions (`VARCHAR(20)`, `VARCHAR(50)` and a text comparison)	Explicit casts with a comment recording why they must stay (`docs/business-workflow-verification-2026-10-30.md` §2)
D-02	Every model-created case was `high` priority and the high-priority threshold was dead configuration; the queue rendered "high 0.00"	Browser verification of the review screen	Priority computed from the review threshold instead of the high-priority threshold; the policy block published `null`	One shared `reviewPriority` function used by both the registration response and the queue; the real value published (`docs/frontend-verification-2026-10-30.md` §1)
D-03	The `risk_level: 'critical'` styling path was unreachable	Browser verification	The API only ever emitted `'review'`	The API keeps `risk_level: 'review'` and publishes `priority` and `exact_identifier_matches` instead
D-04	A second QR scan in the same tab showed the previous result	Headless-Chrome QR harness	The page verified once on mount; a hash-only change does not remount	Re-verify on `hashchange`, with the token captured during render so React StrictMode's double effect does not break it
D-05	Every QR scan failed on the deployed site (404)	Production re-check through the deployed SPA	`fetch("/api/verify")` relative to the static host, which deliberately does not proxy `/api/`	Route the call through the axios client that honours `VITE_API_ORIGIN`
D-06	Stored dates of birth scored as non-matching	Integration testing of screening against the database	`pg` returns `DATE` at local midnight, which serialises to the previous calendar day east of UTC	Normalise to `YYYY-MM-DD` at the single predictor boundary (six tests)
D-07	A legitimate registration could be flagged with near certainty at scale	Measured, then simulated	One flag per qualifying pair against the whole registry	Aggregation policy with `top-1` default and counters for suppressed candidates

#	Defect	How it was found	Root cause	Resolution
D-08	`/api/verify` was registered twice, so the limiter and handler could be detached by an edit	Code review recorded as observation O11	Two independent route registrations	One chain — limiter, parser, handler — with a regression test asserting the ordering (`SBMS_O10_O11_IMPLEMENTATION_REPORT.md`)
D-09	Public rate limiting would put all users in one bucket behind a proxy	Code review during the QR phase	Limiter keyed on the socket address	`TRUST_PROXY_HOPS` sets Express `trust proxy`; the default preserves direct-socket behaviour

5.4.8 Proposed and outstanding tests

The following tests have **not** been executed. They are listed separately, as required, and none is reported as a pass anywhere in this chapter.

Table 5.12: Outstanding tests

#	Test	Why it is outstanding	What it would establish
O-01	Phone-camera scan of a printed student QR code and of a printed certificate	Requires a public HTTPS origin and a physical device; the deployment exists but no camera run is recorded	That the artefact works as a physical document, which is the project's stated identification mechanism
O-02	Load test at 500 and 1 000 candidates, and concurrent registrations	No hosted load test or concurrency harness exists	Whether NFR-01 and NFR-13 hold beyond a 50-candidate single-threaded case
O-03	Concurrency test of two simultaneous registrations of the same identifiers	The advisory lock exists; no test exercises it concurrently	That the lock prevents interleaved comparisons
O-04	Print-output verification of the certificate PDF and of the exported report	Only generated bytes are covered	That the printed documents are legible and correctly laid out
O-05	Accessibility audit of the public pages and staff screens	No automated or manual accessibility review exists	Whether the interface meets a stated accessibility standard
O-06	Cross-browser testing (Firefox, Safari, mobile browsers)	Only Chrome was used	NFR-19 beyond Chrome
O-07	Tests asserting the arithmetic of every dashboard counter and report figure	Counters are exercised for authorisation, not arithmetic	That each reported figure matches the underlying data
O-08	Automated test of the eligibility scoring arithmetic	No such test exists	FR-12 beyond the route and role check
O-09	Threshold validation against adjudicated pairs	Requires reviewer-labelled ground truth (§5.5.5)	Whether 0.50 is an appropriate operating point for real records
O-10	Disaster-recovery drill against the **hosted** database	The rehearsal used a local disposable database	That the production recovery procedure works against the managed service
O-11	Behavioural test of the legacy `beneficiaries` module	The router is mounted and readiness depends on it, but no behavioural test exists	Whether the legacy module still does anything useful
O-12	Certificate verification page in a real browser	No script exists for it; covered by API tests only	That the page renders its four states correctly

---

5.5 System evaluation

5.5.1 Evaluation approach

The system was evaluated against three references: the four specific objectives in Chapter One (§1.3.1), the requirement register in Chapter Four, and the problem statement itself. For each reference the evaluation distinguishes four outcomes — *demonstrated*, *partially demonstrated*, *not demonstrated* and *not evaluable with the*

evidence available. A requirement is "demonstrated" only where a test or a recorded observation establishes it; implementation alone is not demonstration.

5.5.2 Evaluation against the specific objectives

Table 5.13: Achievement of the specific objectives

Objective (Chapter One §1.3.1)	Evidence of achievement	Verdict
(i) Investigate the existing challenges of manual registration and tracking in Kenya's government-funded welfare programmes	Document analysis of Auditor-General, Controller of Budget and NG-CDF sources in Chapter Two, and the requirements derived from it in Chapter Four §4.2.1. No primary data collection was undertaken	**Partially achieved** — the secondary evidence base is documented, but the investigative dimension rests on published audit findings rather than field research
(ii) Design the architecture, database schema and machine-learning pipeline, incorporating a Random Forest classifier	Chapter Four §4.4–§4.7 presents the delivered architecture, schema (12 tables, 56 constraints) and pipeline. The classifier **shipped as XGBoost over six features**, not Random Forest over four; the Random Forest exists only as an unevaluated research candidate	**Achieved with a documented design change** — the substitution must be acknowledged rather than explained away
(iii) Implement a web-based system supporting digital registration, unique identifier generation, QR-based identification and role-based access control	Registration (FR-05), server-generated UUID keys and 64-hex QR tokens (FR-20), two-role access control enforced in middleware and pinned by tests (FR-04), and a deployed instance at a public HTTPS origin — covered by automated tests plus browser and deployment checks	**Achieved**
(iv) Validate and test the system to evaluate its effectiveness in detecting duplicate records and improving beneficiary tracking	151 backend tests, 43 native tests, 3 smoke checks, 33 + 7 Python tests, 43 browser checks and deployment checks. Detection effectiveness **against real records is not established**: labels are synthetic and no adjudicated pairs exist. Tracking is demonstrated through the award, payment, reconciliation and certificate chain, covered by an eleven-test end-to-end suite	**Partially achieved** — validated as an artefact and the tracking chain demonstrated; detection effectiveness demonstrated only against synthetic data

The honest reading is that two objectives are achieved, one is achieved with a documented design change, and one is partially achieved for a reason that cannot be removed within scope: no legitimately obtainable, adjudicated set of real duplicate/non-duplicate beneficiary pairs exists for this project, so no statement about real-world detection effectiveness follows from its evidence.

5.5.3 Evaluation against functional requirements

Of the thirty delivered functional requirements, **twenty-eight** are demonstrated by at least one executed test that asserts the behaviour, and two are implemented but not demonstrated at the level the requirement specifies:

- **FR-12 (eligibility scoring)** — the route, validation and role restriction are tested, but the point arithmetic and category boundaries are not. Demonstration would need a table-driven test of `calculateEligibility`.
- **FR-25 (dashboard)** — authorisation and synthetic-count reporting are tested; the correctness of each of the nine accountability counters and of the two trend series is not asserted.

Three further requirements are only partially demonstrated: FR-26 (PDF export) is tested for date-range validation but its rendered content is verified by inspection; FR-10 (masking) is tested for the queue response, while the reveal endpoint's audit row is asserted rather than the disclosure workflow exercised in a browser; FR-19 (exceptions) is tested through the discrepancy case in the workflow suite, not for all eleven reason codes.

Two requirements are **not implemented by design** (FR-32 NEMIS integration, FR-33 offline and native mobile) and four recorded gaps remain open (FR-34 candidate blocking, FR-35 QR re-issue, FR-36 token expiry, FR-37 financial transfer). None is a defect against the approved scope: the first two were excluded in the proposal, and the latter two were never claimed.

5.5.4 Evaluation against non-functional requirements

Table 5.14: Non-functional requirement outcomes

NFR	Outcome	Basis
01 Performance (≤ 3 s screening)	Satisfied at the tested size; **not evaluated at registry scale**	22.67 ms for 50 pairs; no larger measurement
02–12 Security, privacy, integrity	Satisfied	Fifteen named tests across `securityMiddleware`, `roleAuthorization`, `verification*`, transaction and certificate suites; masking, redaction and rate limits are asserted
13 Concurrency	**Not evaluated**	Advisory lock present; no concurrent test
14 Usability	**Not evaluated with users**	Implemented and browser-checked; no officer session
15 Maintainability	Satisfied	Shared feature module, ordered migrations, three test frameworks, pinned versions
16 Cost	Satisfied as configuration	Deployed topology inside the approved cost envelope
17 Human oversight	Satisfied	Every decision path is role-gated and audited
18 Recovery	Procedure rehearsed; **no scheduled backup**	`docs/database-recovery-rehearsal-2026-10-31.md`
19 Browser compatibility	Chrome only	No cross-browser record
20 Predictable degradation	Satisfied	503 with no student created when the predictor is unavailable
21 Audit completeness	Satisfied for tested workflows; **no immutability or retention control**	Ten-action audit sequence asserted

5.5.5 Machine-learning evaluation

The model was evaluated on a **synthetic holdout**, and this subsection reports those figures with the limits that make them interpretable. Nothing here is an accuracy claim about Kenyan beneficiary records.

Training and evaluation design. Two independently generated datasets of 12 000 pairs each (6 000 duplicate groups, balanced 6 000/6 000) were used: an internal training set split 80/20 by identity group, and an independent holdout generated from new latent identities and a separate random seed. The holdout was not used for fitting or for parameter tuning beyond model selection between candidate algorithms, and the artefact reports this explicitly (`holdout_used_for_training_or_tuning: false`, `holdout_used_for_model_selection: true`). The selected configuration was `n_estimators 250`, `max_depth 4`, `learning_rate 0.05`, `subsample 0.9`, `colsample_bytree 0.9`, `reg_lambda 1.0`, evaluated at a threshold of 0.50.

Table 5.15: Pair-level performance on the synthetic holdout at the live 0.50 cut-off

Metric	Value
Accuracy	0.9078
Precision	0.9204
Recall	0.8927
F1	0.9063
Average precision (PR-AUC)	0.9727
ROC-AUC	0.9685
Confusion matrix (TP / FP / TN / FN)	5 356 / 463 / 5 537 / 644
Miss rate	10.73%
False-positive rate on the hard-negative pool	7.72%

Threshold behaviour. Because a single threshold is a policy choice rather than a property of the model, the operating point was measured across a range.

Table 5.16: Operating-point trade-off (synthetic holdout)

Cut-off	Precision	Recall	F1	Miss rate	FPR (hard negatives)	Review share
0.30	0.8694	0.9352	0.9011	6.48%	14.05%	53.78%
0.40	0.8985	0.9132	0.9058	8.68%	10.32%	50.82%
0.50 (live)	**0.9204**	**0.8927**	**0.9063**	**10.73%**	**7.72%**	**48.49%**
0.60	0.9404	0.8647	0.9009	13.53%	5.48%	45.97%
0.70	0.9607	0.8270	0.8888	17.30%	3.38%	43.04%
0.80	0.9791	0.7797	0.8681	22.03%	1.67%	39.82%
0.90	0.9899	0.7037	0.8226	29.63%	0.72%	35.54%

F1 is essentially flat between 0.30 and 0.60 (0.9011–0.9065), the best value is 0.9065 at 0.35, and the live 0.50 is 0.0002 below it while having the highest Youden J (0.8155). The measurement therefore does **not** justify moving the production cut-off, and equally does not justify the claim that 0.50 is optimal — it says only that this data cannot distinguish the options in that band.

Where the flags actually come from. The negatives were deliberately hard, and the per-family rates show what kind of case reaches a reviewer.

Table 5.17: Negative families at two cut-offs (synthetic holdout)

Negative family	Pairs	Flagged at 0.50	Flagged at 0.70
`identifier_collision`	1 202	22.21%	11.98%
`common_name_same_school`	1 492	7.84%	2.41%
`twins_or_close_age`	1 150	5.57%	1.57%
`siblings_shared_household`	1 532	0.98%	0.33%
`unrelated_candidate`	624	**0.00%**	**0.00%**
`repeat_application` (positives)	6 000	89.27% recalled	82.70% recalled

The model does not emit noise: genuinely unrelated synthetic pairs never reach the cut-off. What it flags are cases a human would want to see — reused identifiers, twins, same-age siblings, identical common names in one school. That is correct screening behaviour, and it is the reason queue volume has to be governed by candidate selection rather than by declaring the model wrong.

Registration-level consequences. Pair-level precision is not what an administrator experiences, because a registration compares one record against *every* student. The probability of at least one nuisance flag per legitimate registration is `1 - (1 - p)^N`; with `p = 0.35%` (the measured rate on 2 000 random unrelated pairs, `docs/ml-evaluation-baseline-2026-10.md`) at `N = 1 000` that is 96.98%, and the simulation reproduced 97.00%. At cut-off 0.50 the simulation reported, for hard negatives, 970 nuisance flags per 1 000 legitimate registrations at `N = 1 000` and 1 000 at `N = 5 000`; at 0.90 the random-unrelated column collapses to zero but the hard-negative column remains at 999–1 000. Raising the cut-off therefore cannot fix queue volume while the comparison set is the whole registry, because recall falls faster than the queue shrinks (0.8927 → 0.7037 between 0.50 and 0.90).

The `top-1` policy also has a second cost, which the aggregation tests do not measure: if a nuisance candidate outranks the true duplicate, the reviewer is shown the wrong pair and the real duplicate is suppressed. The simulation reports clean detection falling from 67.6% at `N = 100` to 36.6% at `N = 5 000`, with masking rising from 23.8% to 52.6%; the controller ranks exact identifier agreement above score, which rescues identifier-verified duplicates and is not modelled in the simulation, so those masking figures are pessimistic but not negligible.

Table 5.18: Candidate-generation projection (only lever that changes the order of magnitude)

Candidate key	Block size at N = 1 000	Block size at N = 5 000	Duplicates retained
Same surname **and** same school	0.99	4.95	**16.47%**
Any shared name token	29.85	149.25	**80.52%**

The surname-and-school key looks excellent on volume and is unusable in practice: it silently discards 83.5% of duplicates. Name-token blocking retains 80.52% while cutting the comparison set by roughly 33× at `N = 1 000`, and the projected nuisance rate behind it ranges from 9.94% to 90.90% of registrations at that size — a real improvement, and an honest measure of how much remains unknown.

Class imbalance and the cost of errors. The evaluated label mix is balanced 50/50, which is not the prevalence expected operationally; average precision therefore depends on the synthetic 25% positive prevalence in the research dataset and on 50% in the production holdout. The asymmetry that matters is operational: a false positive costs reviewer time bounded by the aggregation policy, while a false negative leaves a duplicate beneficiary unflagged. The project did not fix a miss-tolerance policy (decisions D1–D5 remain unanswered), so the choice between those costs was deferred rather than made — recorded in Chapter Six as a required decision.

What this evaluation does not establish. It does not establish precision, recall or miss rate on real beneficiary records; it does not establish the false-positive rate of the applicant population; it does not establish that 0.50 is the right cut-off; it does not cover duplicate patterns other than `repeat_application`; and it does not establish generalisation beyond the simulator. Those exclusions are the project's own, stated in `docs/ml-decision-sheet-2026-10-30.md` §8, and they are the reason the chapter reports a pipeline result rather than a detection capability.

5.5.6 Usability and user acceptance

User acceptance testing was **not conducted with real users**. A UAT plan exists (`docs/uat-test-plan.md`) with five scenarios — login, registration with duplicate detection, duplicate review, disbursement control, and dashboard/reporting — each marked `PASS`, and an audit-trail check. That document cannot be presented as participant-based acceptance evidence, for three reasons that must be recorded rather than smoothed over: it names no participants, roles, dates or locations; it records no observations or free-text feedback; and one of its scenarios still states that the backend does not call the Python predictor, which the implementation clearly does (Chapter Four §4.4.4). It is best read as an author-executed scripted walkthrough of the happy paths, superseded in evidential value by the browser harnesses in §5.4.6, which record what the rendered application actually did.

Consequently NFR-14 (usability) and any claim about user acceptance are **REQUIRES INPUT**: participants, tasks, findings and consent arrangements must be supplied by the author if an acceptance study is to be reported.

5.6 Discussion of results and limitations

5.6.1 What the implementation achieved

The system built is a working, deployed, role-separated web application in which a beneficiary is registered once, screened against the existing registry and issued a QR-verifiable identity; a possible duplicate becomes a review case that only an administrator can close; and money cannot be approved or paid to a student whose case is open or confirmed. Around that spine sit award submission with mandatory verification evidence, payment records that distinguish issuance from settlement, receipt acknowledgement that preserves a discrepancy instead of hiding it, an exception list derived from the data, an audited disclosure path for beneficiary identifiers, and a QR-verifiable award certificate that reports its own staleness.

Three capabilities deserve emphasis because they exceed what the proposal promised and make the artefact believable as an accountability tool rather than a record store: the certificate snapshot that survives a later award change; the derived exception rules that make under-payment and missing acknowledgement visible; and the masking-plus-audit design that lets an administrator see enough to judge a case without turning the queue into a personal-data browser.

5.6.2 How the system addresses the problem in Chapter One

The problem was that beneficiary records are held on paper in independent offices, that the same learner can appear more than once, and that payments cannot be traced to a verified student. Against that, the delivered system contributes a single registry with a unique identifier per student; a screening step that surfaces near-

duplicates exact matching would miss, with a human decision required; a payment chain in which every step is attributable to a named user and carries evidence; and a verification artefact a school or guardian can check without gaining access to the beneficiary's data.

It does **not** address the root cause of cross-constituency duplication, because the system is single-tenant: it screens within one registry, and a student registered in two different constituency deployments would still not be detected. That limitation is inherent to the scope (no government data-sharing agreement, no NEMIS integration) and must not be glossed: within-registry detection is a genuine improvement over paper, but it is not national deduplication.

5.6.3 What the testing establishes, and what it does not

The executed evidence establishes that: the access-control model behaves as specified for every role-sensitive route tested; the two payment gates cannot be bypassed through the documented API, including when the student appears as the *matched* rather than the *new* record; database constraints independently enforce the same rules; the public endpoints disclose only what they claim, and refuse without distinguishing "unknown" from "malformed"; the predictor is correctly wired, versioned and fail-closed; and the deployed configuration behaves like the tested configuration for the checks run, after one genuine production defect was found and fixed.

It does **not** establish that the deployed application behaves correctly for a real officer working with real documents: there is no usability observation, no phone-camera scan, no print check, no load test and no cross-browser evidence. Nor does it establish that the model's performance on real records resembles its performance on the simulator.

5.6.4 Technical and operational limitations

1. **Candidate generation is unindexed.** Every registration is scored against every existing student, and the administrative rescan is quadratic. This bounds the registry size at which the system is usable in practice.
2. **The predictor is a single child process.** One artefact, one process, one lifetime bound to the API; a crash fails current registrations and momentarily disables registration, and it accumulates memory per test file, which is the documented cause of the intermittent failures in §5.4.3.
3. **Rate limiting is per process.** Counters reset on restart and are not shared between instances, so horizontal scaling weakens the protection.
4. **Threshold changes are not durable.** `PUT /api/settings/threshold` alters in-memory values that a restart discards, with no persisted configuration record beyond the audit entry.
5. **No backup schedule or retention policy.** Recovery is a rehearsed manual procedure; audit rows are append-only by convention only; nothing is ever deleted, which is a privacy liability as well as a storage one.
6. **Two partially overlapping notions of "beneficiary"** exist (the `students` model and the legacy `beneficiaries` table), with no reconciliation and an unresolved `scholarship_amount` meaning.
7. **The dead rule engine** in `duplicateCheck.js` remains exported and is exercised only by tests — a maintenance hazard in a codebase whose central concept is duplicate scoring.
8. **No individual session revocation** and no multi-factor authentication.

5.6.5 Data and machine-learning limitations

The central limitation is that **no adjudicated real-world label set exists**. Everything measured is a property of a simulator: the positives are one duplicate pattern (`repeat_application`); the negatives are designed rather than observed; the balance is not operational prevalence; and one artefact at one seed produced every number, so a retrain invalidates the report. The decisions that would convert these measurements into a defensible operating point — flag budget, review capacity, miss tolerance, cross-school comparison, adjudication owner — are recorded as unanswered in `docs/ml-evaluation-policy.md` and remain unanswered.

A second limitation is specific to the data the system can collect: the registry uses five school *names* as identifiers because no authoritative school-code list exists, so the model's `school_match` feature compares names. Until an

authoritative code list and a name-to-code mapping are supplied, the feature is weaker than its name suggests (`docs/ml-integration-and-school-identifiers.md`).

5.6.6 Security, deployment and integration limitations

- The deployment is **not continuously available**: the managed database is stopped between demonstrations to stay inside a student credit budget, so availability is a scheduled property, not a service level.
- The system is **not integrated** with NEMIS, a national identifier service, a bank or a mobile-money operator, and it neither executes nor confirms a financial transfer.
- Receipt acknowledgement is **officer-assisted**; independent school confirmation does not exist.
- Certificate verification is live and therefore depends on the database being up; a printed certificate cannot be checked while the deployment is stopped, a trade-off that favours truthfulness over offline convenience.
- The public rate limits are in-process, and `TRUST_PROXY_HOPS` must be configured correctly behind a proxy or all public users share one budget (defect D-09).

5.6.7 Reliability of the evidence itself

Two project records conflict with the implementation and are named here so that the report does not inherit their errors. `docs/verification.md` states that "a repository reference check found no current callers" of the predictor and that controller decisions still use their own scoring; that was true of an earlier revision and is false now (`students.controller.js:69`). `docs/uat-test-plan.md` both asserts that the predictor is not called and marks all five scenarios `PASS` without participant records. Both documents are superseded by `docs/FINAL_TEST_REPORT.md`, `docs/business-workflow-verification-2026-10-30.md` and the suite inventory in §5.4.3. `docs/FINAL_TEST_REPORT.md` itself contains two generations of result (128 tests on 2026-10-31, 151 on 2026-11-01), which is why this chapter always names the date of a count.

5.7 Chapter summary

This chapter reported an implementation whose technical spine is sound and whose evidence has clear edges. On the implementation side: a three-tier system with a supervised Python scorer, a two-role access model re-validated against the database on every request, database-enforced integrity rules around awards and payments, two independent payment gates, QR and certificate verification separated technically as well as conceptually, and a deployment whose image cannot be published if the model cannot score.

On the evidence side: 151 automated backend tests, 43 native tests, 3 out-of-process smoke checks, 40 Python tests, 43 browser checks and a set of live deployment checks, with nine defects found and fixed during testing — one of them, payment clearance, a total functional failure that no earlier test had covered, and one a production-only routing defect that made every QR scan fail on the deployed site.

On the evaluation side: two objectives achieved, one achieved with a documented classifier substitution, and one partially achieved because the evidence needed to establish real-world duplicate-detection effectiveness does not exist within the project's scope. The model's measured performance (F1 0.9063, precision 0.9204, recall 0.8927 at 0.50 on a synthetic holdout) is a property of the simulator, and the operating point is unvalidated because there are no adjudicated labels; the more consequential finding is that candidate generation, not the threshold, governs the practical review load.

Chapter Six draws conclusions from exactly this evidence, and confines its recommendations to what the limitations above make necessary.